Árvores balanceadas para encomendas

Amazon Engineering Challenge

AVL e Rubro-Negra

Guilherme Soares
Estruturas de Dados Básicas II

O problema

Na Black Friday, identificadores de encomendas precisam ser cadastrados, localizados e removidos.

Uma árvore de busca comum pode ficar parecida com uma lista se os dados chegarem em ordem.

Objetivo: implementar duas árvores balanceadas e comparar seus resultados.

Por que balancear?

Em uma árvore de busca, valores menores ficam à esquerda e maiores à direita.

Se a altura cresce demais, o caminho até um elemento também cresce.

AVL e Rubro-Negra mantêm altura O(log n), garantindo busca, inserção e remoção O(log n).

Solução desenvolvida

Programa em C++17, com escolha da árvore no início.

Operações: inserir, buscar, remover, listar em ordem e visualizar.

Identificadores repetidos são recusados. Busca e remoção de ausentes mostram uma mensagem.

A visualização da Rubro-Negra identifica nós pretos [P] e vermelhos [V].

AVL: altura e rotação

A AVL mantém a diferença entre as alturas das subárvores de cada nó entre -1 e 1.

Exemplo: ao inserir 30, 20 e 10, ocorre uma rotação à direita em 30.

Antes: 30 com filho 20, que tem filho 10.
Depois: 20 é a raiz, com 10 à esquerda e 30 à direita.

Rotações duplas são usadas quando necessário.



Rubro-Negra: cores e ajustes

A raiz e as folhas sentinelas são pretas. Um nó vermelho não pode ter filho vermelho.

Todo caminho de um nó até uma folha sentinela tem a mesma quantidade de nós pretos.

Ao inserir 30, 20 e 10, a correção usa rotação e mudanças de cor: 20 fica preto na raiz, com 10 e 30 vermelhos.



Demonstração das operações

Nas duas árvores: inserir 30, 20 e 10; buscar 20; remover 30; listar em ordem.

Busca: 20 encontrado.
Após remover 30, percurso em ordem: 10 20.

As saídas completas da execução estão em resultados/ no repositório.

Testes de funcionamento

Casos: árvore vazia e unitária, duplicados, busca ausente, remoção ausente, folhas e nós com um ou dois filhos.

20.000 operações pseudaleatórias em cada árvore, comparadas a um conjunto de referência.

Após cada operação, são verificadas a ordem e as propriedades da estrutura. Todos os testes passaram.

Como a comparação foi feita

Para cada cenário, inserimos 1.000 identificadores e removemos os primeiros 500 da mesma sequência.

Cenários: crescente, decrescente e embaralhado.

Altura conta nós: árvore vazia = 0. Rotações são acumuladas desde o início; uma rotação dupla conta como duas.

Resultados após as inserções

Cenário	Altura AVL / RN	Rotações AVL / RN
Crescente	10 / 17	990 / 983
Decrescente	10 / 17	990 / 983
Embaralhado	12 / 12	662 / 564

Ajustes de cor da RN e nós rebalanceados da AVL são medidas diferentes e não são comparados diretamente.

Resultados após 500 remoções

Cenário	Altura AVL / RN	Rotações AVL / RN
Crescente	10 / 15	1238 / 1232
Decrescente	10 / 15	1238 / 1232
Embaralhado	11 / 11	874 / 790

Os totais de rotações incluem inserções e remoções.

Recomendação e limites

Para priorizar altura menor e caminhos de busca mais curtos neste protótipo, a AVL é uma escolha razoável.

A Rubro-Negra fez menos rotações nos três cenários e é uma alternativa a considerar quando atualizações são frequentes.

Não medimos tempo nem usamos uma carga real da Amazon. Mais testes seriam necessários para decidir em produção.

Referências e código

Código, instruções, testes e resultados:
https://github.com/guilhermesoaresf/amazon-arvores-balanceadas

Enunciado da Unidade II, IMD0039.

Cormen et al. Introduction to Algorithms, 3ª edição.

Open Data Structures: https://opendatastructures.org/